

FPGA-Based Zombie Shooting Game with VGA Display and Audio System

b12901193 Chin-Yen, Pan
b12901166 Yin-Liang, Chen
b12901030 Yu-Siang Huang

December 27, 2025

Contents

1	File Structure	4
2	Topic	4
2.1	Key Advantages	5
3	Overview	6
3.1	System Architecture	6
3.2	Hardware Platform	7
3.3	Main Features	7
4	Methods	7
4.1	Algorithms	8
4.1.1	K-means Color Quantization	8
4.1.2	Image Overlay with Bounds Checking	9
4.2	Memory Management	10
4.2.1	Index-Based Storage	10
4.2.2	Palette System	10
4.2.3	Bounds Management	10
4.2.4	Memory Hierarchy	11
4.2.5	Address Mapping	11
5	Module Introduction	12
5.1	VGA Display Modules	12
5.1.1	VGA Controller 720p	12
5.1.2	VGA Image Compositor	14
5.2	Game Logic Modules	18
5.2.1	Start Game Detector	18
5.2.2	Random Position Generator	19
5.3	Audio Modules	20
5.3.1	Audio Control	20
5.3.2	AudPlayer	21
5.4	Input Control Modules	22
5.4.1	UART Receiver	22
5.4.2	IMU Parser	23
5.4.3	Velocity Integrator	23
5.4.4	Position Controller	24
6	System Architecture	25
6.1	Overall System Architecture	25
6.2	Detailed Rendering Pipeline	26
7	Finite State Machine (FSM) Diagrams	27
8	Implementation Results	28
8.1	Resource Utilization	28

9	Timing Analysis	30
9.1	Overview	30
9.2	Worst-Case Setup Summary (Slow 1200mV 85C)	30
9.3	Multicorner Summary	30
9.4	Analysis and Impact	31
10	Challenges and Solutions	31
10.1	Transparency Detection Optimization	31
10.1.1	Challenge: Efficient Transparency Detection for Zombie Sprites	31
10.1.2	Attempt 1: Sequential Pixel Index Checking	31
10.1.3	Attempt 2: 2D Boolean Array Lookup	32
10.1.4	Solution: Pre-computed Column-Based Bounds in BRAM	32
10.2	Multi-Sprite Rendering Performance	33
10.2.1	Challenge: Rendering 20 Simultaneous Zombies	33
10.2.2	Solution: Candidate Selection and Bounds Optimization	33
10.3	Memory Resource Constraints	33
10.3.1	Challenge: High Memory Utilization	33
10.3.2	Solution: Efficient Memory Architecture	33
10.4	IMU Data Processing and Drift Management	34
10.4.1	Challenge: Sensor Drift and Noise	34
10.4.2	Solution: Calibration and Intelligent Drift Correction	34
10.5	VGA Timing Compensation	34
10.5.1	Challenge: BRAM and SRAM Read Latency in VGA Pipeline	34
10.5.2	Solution: Predictive Coordinate Generation	35
11	Experience of the Project	35
11.1	Technical Learning Outcomes	35
11.1.1	System-Level Design	35
11.1.2	Memory System Design	35
11.1.3	Pipeline Design and Timing	35
11.1.4	Real-Time System Constraints	36
11.2	Problem-Solving Skills	36
11.3	FPGA Development Workflow	36
11.4	Interdisciplinary Integration	37
11.5	Project Management and Teamwork	37
11.6	Real-World Application Insights	37
11.7	Challenges and Growth	38
11.8	Conclusion	38
A	Appendix - Project Resources	39
A.1	GitHub Repository	39
A.2	Demo Video	39

1 File Structure

```
team06_final/
├── team06_final_report.pdf
├── team06_final_presentation.pdf
├── src/
│   ├── DE2_115/
│   │   ├── DE2_115.sv
│   │   ├── DE2_115.sdc
│   │   └── DE2_115.qsf
│   ├── IMU/
│   │   ├── uart_rx.sv
│   │   ├── imu_parser.sv
│   │   └── velocity_integrator.sv
│   ├── zombie_position_control/
│   │   ├── Position_Controller.sv
│   │   └── Random_Position_Generator.sv
│   ├── Audio/
│   │   ├── audio_control.sv
│   │   ├── AudPlayer.sv
│   │   └── I2cInitializer.sv
│   ├── VGA_720p/
│   │   ├── VGA_Controller_720p.sv
│   │   └── VGA_Compositor_720p.sv
│   └── others/
│       ├── Debounce.sv
│       ├── HexTo7Seg.sv
│       └── Start_Game_Detector.sv
├── BRAMs/
│   ├── aim/
│   │   └── aim.qip, aim.v, aim sprite data
│   ├── audio/
│   │   └── bgm.mif, killed.mif, BGM and sound effects
│   ├── background/
│   │   └── background_palette.v, start_palette.v
│   ├── end/
│   │   └── win_palette.v, lose_palette.v, retry_palette.v
│   └── zombies/
│       ├── zombie_05x.v, zombie_06x.v, zombie_07x.v, zombie_08x.v, zombie_09x.v,
│       │   zombie_1x.v
│       ├── zombie_*x_bound.v, zombie_*x_bound2.v (transparency bounds)
│       └── zombie_1x_palette.v (used by every zombies)
```

2 Topic

This project presents a fully functional, real-time zombie shooting game implemented entirely on an FPGA platform. The system delivers an immersive gaming experience through high-definition 720p VGA graphics, dynamic audio feedback, and intuitive motion-

based aiming control using an Inertial Measurement Unit (IMU). Players engage in an action-packed survival scenario where they must eliminate waves of zombies approaching from various positions on screen, with the game dynamically adjusting difficulty based on performance.

The core innovation lies in the seamless integration of multiple complex subsystems operating in parallel: a pixel-perfect VGA display engine capable of rendering multiple animated sprites simultaneously, a sophisticated audio playback system providing background music and sound effects, and a responsive motion control interface that translates physical movements into precise on-screen aiming. The game features intelligent zombie spawning algorithms, real-time collision detection, dynamic sprite scaling for depth perception, and comprehensive game state management including win/lose conditions and scoring mechanisms.

2.1 Key Advantages

The FPGA-based implementation offers several significant advantages over traditional software-based game systems:

- **Ultra-Low Latency:** Hardware-accelerated rendering and processing eliminate software overhead, providing near-instantaneous response to player inputs. The parallel architecture ensures that VGA timing, audio playback, and game logic execute simultaneously without interference.
- **Real-Time Performance:** The dedicated hardware pipeline guarantees consistent 60 frames per second rendering at 1280x720 resolution, with deterministic timing for all game mechanics. Unlike software implementations that may suffer from frame drops or timing jitter, this system maintains precise synchronization across all subsystems.
- **Memory Efficiency:** By utilizing palette-based color indexing and strategic use of SRAM for large background images and BRAM for frequently accessed sprite data, the system achieves high-quality graphics while minimizing memory footprint. Multiple zombie sprites at different scales are stored efficiently, enabling up to 20 simultaneous zombies on screen.
- **Deterministic Behavior:** All game logic executes in fixed clock cycles, ensuring predictable and reproducible gameplay. This determinism is crucial for game mechanics such as collision detection, zombie movement patterns, and audio synchronization.
- **Modular Architecture:** The system is designed with clear module boundaries, making it easy to modify, extend, or debug individual components. Each subsystem (VGA, audio, input control, game logic) operates independently while communicating through well-defined interfaces.
- **Power Efficiency:** FPGA implementations can be more power-efficient than general-purpose processors for dedicated tasks, as only the required logic is active, and there is no instruction fetch overhead.

- **Educational Value:** This project demonstrates practical application of digital design principles including state machines, pipeline architectures, memory interfaces, clock domain crossing, and real-time system design.

The system successfully demonstrates that complex, interactive multimedia applications can be efficiently implemented on FPGA platforms, providing a foundation for future embedded gaming systems, interactive displays, or real-time graphics applications.

3 Overview

This project, titled "FPGA-Based Zombie Shooting Game", is a real-time interactive survival game that leverages the high-performance parallel processing capabilities of FPGAs. The game tasks players with defending against waves of zombies using an IMU-based motion control system. By combining high-definition VGA graphics, synchronized audio effects, and low-latency input processing, the system provides a compelling demonstration of complex system-on-chip (SoC) design on an FPGA.

3.1 System Architecture

The system follows a modular architecture where the top-level module (DE2_115) orchestrates the communication between several key functional blocks. This modularity ensures efficient data flow and clear separation of concerns:

- **Display Subsystem:** Centered around the `VGA_Controller_720p` and `VGA_Image_Overlay_Combined` modules. It manages a 6-stage pipeline to composite multiple layers (background, sprites, and UI) in real-time.
- **Input Subsystem:** Processes motion data from an external **LSM6DSL IMU** mounted on an **STM32L4S5VIT6** microcontroller. The STM32 reads raw sensor data via I2C and transmits orientation information to the FPGA through UART. A `UART_RX` module on the FPGA receives this data, which is then parsed by the `IMU_Parser` and integrated by the `Position_Controller` to drive the on-screen aim cursor.
- **Audio Subsystem:** Managed by the `audio_control` module, it handles background music (BGM) playback and triggers specific sound effects (SFX) for game events like shooting or zombie deaths.
- **Game Logic:** Implements the core mechanics, including random zombie spawning (`Random_Position_Generator`), collision detection (within the compositor), life tracking, and win/lose condition management.
- **Memory Hierarchy:** Balanced use of External SRAM (for high-resolution backgrounds) and On-chip BRAM (for localized sprite indices, palettes, and transparency bounds).

3.2 Hardware Platform

The system is developed and deployed on the **Terasic DE2-115 Development Board**, featuring a Cyclone IV EP4CE115 FPGA. The following hardware resources and peripherals are utilized:

- **Cyclone IV FPGA:** The heart of the system, implementing all logic and on-chip memory.
- **2MB SRAM:** Used as a frame buffer and storage for large indexed images (1280x720).
- **VGA (ADV7123):** High-speed video DAC driving the 720p (74.25MHz) display output.
- **Audio Codec (WM8731):** Provides high-quality audio output for game music and effects.
- **LSM6DSL IMU:** A 6-axis motion tracking device (accelerometer and gyroscope) used for orientation sensing.
- **STM32 L4S5VIT6:** Acts as an intelligent sensor hub, performing initial data acquisition from the IMU and bridging it to the FPGA via UART.
- **GPIO & Buttons:** Used for trigger inputs (shooting), hardware resets, and diagnostic controls.

3.3 Main Features

Key features of the implementation include:

- **720p Real-Time Graphics:** Crisp 1280x720 resolution rendered at a consistent 60 FPS without software-related jitter.
- **3D Depth Perception:** Dynamic scaling of zombie sprites (from 0.5x to 1.0x) based on their vertical position, simulating a 3D environment.
- **Responsive Motion Aiming:** Low-latency translation of IMU orientation to target coordinates, providing a "light-gun" style experience.
- **Multi-Sprite Rendering:** Capability to render up to 20 independent zombies simultaneously with individual states and animations.
- **Interactive Audio:** Seamless blending of background survival-themed music with real-time sound effects for shooting and zombie eliminations.
- **Robust Game States:** Integrated flow from a start screen to active gameplay, including difficulty scaling and terminal win/lose states.

4 Methods

This section describes the key algorithms and memory management techniques employed in the system to achieve efficient image processing and rendering.

4.1 Algorithms

4.1.1 K-means Color Quantization

The system uses K-means clustering algorithm to reduce the color space of source images from 24-bit RGB (16.7 million colors) to 8-bit indexed color (256 colors). This compression technique is essential for fitting large images into limited FPGA memory resources while maintaining acceptable visual quality.

Algorithm Overview The K-means quantization process works as follows:

1. **Image Preprocessing:** Source images are loaded in RGBA format, where the alpha channel (A) is used to identify transparent pixels.
2. **Transparent Pixel Handling:** All pixels with alpha channel value of 0 are automatically assigned index 0, which maps to white (255, 255, 255) in the palette. This white color serves as the transparent key.
3. **Cluster Initialization:** The algorithm creates K-1 clusters (where K=256) from all non-transparent pixels. The remaining cluster (index 0) is reserved for transparency.
4. **K-means Clustering:** The scikit-learn KMeans algorithm is applied to the RGB values of all opaque pixels, finding optimal cluster centers that minimize the sum of squared distances from pixels to their nearest cluster center.
5. **White Color Avoidance:** If any cluster center results in pure white (255, 255, 255), it is automatically adjusted to (254, 254, 254) to avoid conflicts with the transparency key.
6. **Index Assignment:** Each pixel is assigned to its nearest cluster center using Euclidean distance in RGB space:

$$\text{distance} = \sqrt{(R - R_c)^2 + (G - G_c)^2 + (B - B_c)^2} \quad (1)$$

where (R_c, G_c, B_c) are the cluster center coordinates.

7. **Palette Generation:** The final palette consists of 256 entries: index 0 = white (transparent), indices 1-255 = K-means cluster centers converted to RGB565 format.

RGB565 Conversion The palette colors are stored in RGB565 format (16 bits) to match the BRAM data width:

$$\text{RGB565} = (R \gg 3) \ll 11 \mid (G \gg 2) \ll 5 \mid (B \gg 3) \quad (2)$$

This format allocates 5 bits for red, 6 bits for green, and 5 bits for blue, providing a good balance between color accuracy and memory efficiency.

Output Format The quantization process generates three types of output files:

- **Index Map (.bin/.mif):** Binary file containing 8-bit pixel indices, stored row by row
- **Palette File (.mif):** Memory initialization file containing 256 RGB565 color values
- **Preview Image (.png):** Reconstructed image for visual verification

4.1.2 Image Overlay with Bounds Checking

To properly composite multiple image layers (background, zombies, aim cursor) without artifacts, the system implements sophisticated bounds checking and transparency handling.

Bounds Calculation For each sprite (zombie), the system pre-computes transparency bounds to optimize rendering performance. The bounds calculation algorithm:

1. **Column-wise Scanning:** For each column (X coordinate) in the sprite, the algorithm scans from top to bottom (Y coordinate).
2. **Min/Max Detection:** For each column, it finds:
 - **Min Y:** The first (topmost) non-transparent pixel (index $\neq 0$)
 - **Max Y:** The last (bottommost) non-transparent pixel (index $\neq 0$)
3. **Bounds Storage:** The result is stored as a 16-bit value per column: $[15:8] = \text{Max Y}$, $[7:0] = \text{Min Y}$
4. **Full Transparent Columns:** If a column contains only transparent pixels, Min Y = 0xFF and Max Y = 0x00, indicating the entire column should be skipped

Overlay Rendering Algorithm The image compositor uses a multi-stage pipeline to determine the final pixel color:

1. **Stage 0 - Candidate Selection:** For each pixel coordinate (h_count, v_count), determine which sprites might overlap this position based on their bounding boxes.
2. **Stage 1 - Bounds Check:** For each candidate sprite, check if the current pixel is within the sprite's bounds and within the non-transparent region (using pre-computed bounds).
3. **Stage 2 - Priority Resolution:** If multiple sprites overlap the same pixel, determine rendering priority (typically: aim cursor > zombies > background).
4. **Stage 3 - Pixel Index Fetch:** Read the pixel index from the appropriate memory (SRAM for background, BRAM for sprites).
5. **Stage 4 - Palette Lookup:** Convert the 8-bit index to RGB565 color using the palette BRAM.
6. **Stage 5 - RGB Conversion:** Convert RGB565 to RGB888 for VGA output, applying any special effects (hit flash, grayscale for dying zombies).

Transparency Handling Transparent pixels (index = 0x00) are handled by:

- Skipping palette lookup for transparent pixels
- Falling through to the next layer (background) when a sprite pixel is transparent
- Using bounds information to avoid unnecessary memory accesses for fully transparent regions

4.2 Memory Management

4.2.1 Index-Based Storage

The system uses an 8-bit index-based storage scheme to minimize memory requirements:

- **Background Images:** Stored in external SRAM as 8-bit indices (1 byte per pixel)
- **Sprite Images:** Stored in BRAM as 8-bit indices (1 byte per pixel)
- **Memory Savings:** A 1280×720 image requires only 921,600 bytes (900 KB) instead of 2,764,800 bytes (2.7 MB) for 24-bit RGB
- **Address Calculation:** Pixel address = $Y \times \text{WIDTH} + X$, stored in row-major order

4.2.2 Palette System

The palette system provides color lookup for indexed images:

- **Palette Storage:** Each palette is stored in BRAM as 256 entries of 16-bit RGB565 values
- **Memory Layout:** Palette BRAM address = pixel index (0-255), data = RGB565 color value
- **Multiple Palettes:** Different palettes are used for different image types:
 - Background palette: For start screen and game background
 - Zombie palette: For zombie sprites at different scales
 - End game palettes: For win/lose/retry screens
- **Palette Lookup:** The conversion from index to color is a single-cycle BRAM read operation

4.2.3 Bounds Management

To optimize rendering performance, the system maintains bounds information for sprites:

- **Pre-computed Bounds:** Transparency bounds are calculated once per sprite and stored in registers
- **Bounds Format:** For each X coordinate, bounds store [Max Y, Min Y] as a 16-bit value

- **Bounds Usage:** During rendering, bounds are checked before accessing sprite pixel data:
 - If current $Y < \text{Min } Y$ or $Y > \text{Max } Y$: Skip sprite pixel fetch (column is transparent at this Y)
 - If $\text{Min } Y \leq Y \leq \text{Max } Y$: Fetch sprite pixel and check if $\text{index} = 0$ (pixel-level transparency)
- **Performance Benefit:** Bounds checking eliminates unnecessary memory accesses for transparent regions, reducing memory bandwidth and improving timing closure

4.2.4 Memory Hierarchy

The system employs a hierarchical memory architecture optimized for different access patterns:

- **SRAM (External):**
 - Capacity: 2MB ($1\text{M} \times 16\text{-bit words}$)
 - Usage: Large background images ($1280 \times 720 = 921,600$ bytes)
 - Access: Sequential, predictable pattern (row-by-row scanning)
 - Latency: 2 clock cycles
- **BRAM (On-chip):**
 - Capacity: Multiple $256 \times 8\text{-bit}$ blocks for sprites, $256 \times 16\text{-bit}$ blocks for palettes
 - Usage: Frequently accessed sprite data and palette lookups
 - Access: Random access for sprite rendering
 - Latency: 1 clock cycle
- **Register Arrays:**
 - Usage: Bounds information, game state, zombie positions
 - Access: Immediate (combinational)
 - Latency: 0 cycles (combinational logic)

4.2.5 Address Mapping

The system uses different address mapping strategies for different memory types:

- **SRAM Address:**

$$\text{SRAM_ADDR} = Y \times \text{WIDTH} + X \quad (3)$$

Since SRAM is 16-bit wide, byte address is divided by 2, and the lower bit selects high/low byte.

- **BRAM Address (Sprites):**

$$\text{BRAM_ADDR} = (Y - \text{sprite_y}) \times \text{sprite_width} + (X - \text{sprite_x}) \quad (4)$$

where sprite coordinates are relative to the sprite's top-left corner.

- **BRAM Address (Palette):**

$$\text{Palette_ADDR} = \text{pixel_index} \quad (5)$$

Direct mapping from pixel index to palette entry.

5 Module Introduction

5.1 VGA Display Modules

5.1.1 VGA Controller 720p

Module Goals The VGA Controller 720p generates timing signals for 1280x720@60Hz VGA output at 74.25MHz. It produces sync, blanking signals, and pixel coordinates with a 5-cycle delay pipeline to compensate for BRAM latency.

Detailed Description The VGA Controller 720p module implements a complete VGA timing generator conforming to the VESA standard for 1280x720@60Hz (720p) resolution. This module serves as the foundation of the display system, providing all necessary timing signals and pixel coordinate information to downstream rendering modules.

Timing Parameters The module implements the standard 720p timing parameters as defined in the VESA specification:

- **Horizontal Timing:** Total line length of 1650 pixels, consisting of 1280 active display pixels, 110 front porch, 40 sync pulse, and 220 back porch pixels
- **Vertical Timing:** Total frame length of 750 lines, consisting of 720 active display lines, 5 front porch, 5 sync pulse, and 20 back porch lines
- **Clock Frequency:** 74.25MHz, generated by a PLL from the 50MHz system clock
- **Refresh Rate:** 60Hz, calculated as $74.25\text{MHz} / (1650 \times 750) = 60.0 \text{ Hz}$

Architecture and Design The module employs a clean separation between combinational logic (denoted with ‘_w’ suffix for "write" values) and sequential logic (denoted with ‘_r’ suffix for "register" values). This design pattern improves code readability and facilitates timing analysis.

The core functionality consists of two main counters:

- **Horizontal Counter (h_count_r):** Counts from 0 to 1649 (H_TOTAL - 1), incrementing every clock cycle
- **Vertical Counter (v_count_r):** Counts from 0 to 749 (V_TOTAL - 1), incrementing when the horizontal counter wraps around

BRAM Delay Compensation A critical feature of this controller is its ability to compensate for BRAM read latency. The module calculates active pixel coordinates one cycle ahead of when they are actually needed. Specifically:

- At cycle T, the module calculates the active coordinates (h_active_count_w, v_active_count_w) based on the current counter values (h_count_r, v_count_r)
- These calculated coordinates are registered in the next cycle (T+1) as h_active_count_r and v_active_count_r
- These registered coordinates are then output as o_h_count and o_v_count, which are used by the VGA Image Compositor to fetch pixel data
- This 1-cycle advance compensates for the 1-cycle read latency of BRAM, ensuring pixel data arrives at the correct time for display

The active coordinate calculation logic checks if the current position is within the active display region:

$$h_active_count_w = \begin{cases} h_count_r - H_BACK_PORCH & \text{if } H_BACK_PORCH \leq h_count_r < H \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

Signal Generation The module generates all standard VGA timing signals:

- **HSYNC (o_hsync)**: Horizontal synchronization pulse, active low during the sync period (pixels 1610-1649)
- **VSYNC (o_vsync)**: Vertical synchronization pulse, active low during the sync period (lines 745-749)
- **BLANK_N (o_blank_n)**: Active high during the visible display region, low during blanking periods
- **SYNC_N (o_sync_n)**: Always low for standard VGA operation
- **VGA_CLK (o_vga_clk)**: Pass-through of the 74.25MHz clock to the VGA connector

5-Cycle Delay Pipeline To ensure proper synchronization with downstream processing stages, the module implements a 5-cycle delay pipeline for all output signals. This pipeline:

- Delays HSYNC, VSYNC, BLANK_N, SYNC_N, and active_video signals by 5 clock cycles
- Delays h_count and v_count coordinate outputs by 5 clock cycles
- Ensures that timing signals and pixel coordinates remain synchronized even after multiple pipeline stages in the image compositor

The pipeline uses shift registers to implement the delay:

```

1 hsync_delay <= {hsync_delay[3:0], hsync_int};
2 vsync_delay <= {vsync_delay[3:0], vsync_int};
3 // ... similar for other signals

```

Output Signals The module provides the following outputs to the VGA Image Compositor:

- **o_h_count [10:0]**: Horizontal pixel coordinate (0-1279) within the active display region
- **o_v_count [9:0]**: Vertical pixel coordinate (0-719) within the active display region
- **o_active_video**: High when the current pixel position is within the active display region

These outputs are used by the VGA Image Compositor to determine which pixel data to fetch from SRAM (for background) and BRAM (for sprites), ensuring that the correct image data is displayed at each pixel position.

Reset Behavior Upon reset ($i_rst_n = 0$), all counters and delay pipeline registers are initialized to zero, placing the controller in a known state at the beginning of a frame. The module immediately begins generating valid timing signals after reset is released.

5.1.2 VGA Image Compositor

Module Goals The VGA Image Compositor is the core rendering engine that composites multiple image layers (background, up to 20 zombie sprites, aim cursor, and end-game screens) into a single RGB output stream. It implements a 6-stage pipeline architecture with sprite scaling (0.5x to 1.0x), transparency handling, collision detection, and visual effects. The module manages SRAM and BRAM interfaces with different access latencies and uses bounds checking to optimize rendering performance.

Detailed Description The VGA Image Compositor is a sophisticated 6-stage pipelined rendering engine that composites multiple image layers in real-time. It handles background images from SRAM, up to 20 zombie sprites from BRAM, an aim cursor overlay, and end-game screens, producing a single RGB888 output stream at 74.25MHz.

Architecture Overview The module implements a 6-stage pipeline architecture optimized for memory latency compensation:

- **Layer 0**: Candidate Selection & Address Calculation (Combinational)
- **Layer 1**: Memory Request & SRAM Address Output
- **Layer 2**: Bounds Check & ID Select & SRAM Data Latch
- **Layer 3**: Pixel Index Fetch (BRAM & SRAM Parse)
- **Layer 4**: Palette Fetch
- **Layer 5**: RGB Conversion & Mixing

Input Registration All inputs are registered in the first stage to improve timing closure and reduce combinational path delays. This includes pixel coordinates (h_count , v_count), zombie positions and valid flags, aim position, game state signals, and control signals.

Layer 0: Candidate Selection and Address Calculation This combinational stage performs several critical calculations in parallel:

Background Address Calculation: The module calculates SRAM addresses for background images based on the current pixel position. For the start screen, it uses a 4-bit per pixel format (4 pixels per 16-bit SRAM word), while the game uses 8-bit per pixel format (2 pixels per word). The base address switches between `START_BG_ADDR` (start screen) and 0 (game playing) based on the `i_started` signal.

Aim Cursor Calculation: The aim cursor is a 200×200 pixel overlay centered at the aim position. The module converts center coordinates to top-left corner coordinates using signed arithmetic to handle off-screen positions, then calculates the local pixel address within the aim sprite.

End Game Images: When win or lose signals are active, the module calculates addresses for end-game images (survived, died, retry) stored in SRAM at predefined addresses. These images are positioned at fixed screen coordinates.

Zombie Candidate Selection: The module scans all `MAX_ZOMBIES` (20) zombies and selects up to 4 candidates that overlap the current pixel position. For each candidate, it:

- Determines sprite size based on Y position (0.5x to 1.0x scaling for depth perception)
- Calculates local coordinates within the sprite
- Stores the zombie index, local coordinates, size selector, and valid flag

Kill Target Detection: When the trigger is pressed (`i_killing`), the module captures the aim center position. This position is later checked during rendering to detect zombie hits.

Layer 1: Memory Request and SRAM Address Output This stage initiates memory requests for all required data:

Bounds BRAM Requests: For each of the 4 zombie candidates, the module requests transparency bounds data from BRAM. Since zombies can be at 6 different scales (0.5x to 1.0x), the module maintains separate bounds BRAM instances for each scale. Dual-port BRAMs are used to fetch bounds for 2 candidates simultaneously, requiring 2 BRAM instances per scale (total 12 BRAM instances).

Aim BRAM Request: The aim cursor pixel data is requested from a dedicated BRAM instance.

SRAM Address Output: The calculated background or end-game image address is output to the SRAM interface. The module handles byte selection (`LB_N`, `UB_N`) based on whether the image uses 4-bit or 8-bit pixel format.

Layer 2: Bounds Check and Priority Selection This stage performs bounds checking and selects which sprite to render:

Bounds Checking: For each zombie candidate, the module checks if the current Y coordinate falls within the non-transparent region using the bounds data retrieved in Layer 1. The bounds data format is $[15:8] = \text{Max Y}$, $[7:0] = \text{Min Y}$ for each X coordinate.

Priority Selection: Among all opaque zombies (those passing bounds check), the module selects the one with the lowest Y coordinate (backmost zombie) for rendering. This ensures proper depth ordering.

Zombie Pixel Address Calculation: For the selected zombie, the module calculates the pixel address within the sprite BRAM based on local coordinates and the sprite width (which varies with scale).

Kill Detection: If the current pixel position matches the captured kill target position and a zombie is present, a kill signal is generated. This triggers the zombie's death animation (grayscale flashing effect).

SRAM Data Latch: The SRAM data from Layer 1 (2-cycle latency) is latched for parsing in the next stage.

Layer 3: Pixel Index Fetch This stage retrieves pixel indices from memory:

Zombie Pixel Fetch: The selected zombie's pixel index is requested from the shared zombie sprite BRAM. The address and size selector are output to the top-level module, which routes the request to the appropriate BRAM instance (0.5x to 1.0x).

SRAM Pixel Parsing: The background pixel index is extracted from the latched SRAM data. For 8-bit mode, the module selects the appropriate byte based on the address LSB. For 4-bit mode (start screen), it extracts one of 4 nibbles from the 16-bit word.

Aim Pixel Fetch: The aim cursor pixel data (2-bit) is retrieved from the aim BRAM.

Layer 4: Palette Fetch This stage converts pixel indices to RGB565 colors:

Palette Lookup: Each pixel index (8-bit for background/zombies, 2-bit for aim) is used to address the corresponding palette BRAM. The module maintains separate palettes for:

- Background (start screen and game)
- Zombies (shared across all scales)
- End-game images (survived, died, retry)
- Aim cursor

Transparency Handling: If a zombie pixel index is 0x00 (transparent), the palette lookup is skipped and the background color is used instead.

Layer 5: RGB Conversion and Mixing This final stage produces the RGB888 output:

RGB565 to RGB888 Conversion: Palette colors are stored in RGB565 format (16 bits). The module converts to RGB888 (24 bits) using bit replication:

- Red: 5 bits $\rightarrow$ 8 bits: {rgb565[15:11], rgb565[15:13]}
- Green: 6 bits $\rightarrow$ 8 bits: {rgb565[10:5], rgb565[10:9]}
- Blue: 5 bits $\rightarrow$ 8 bits: {rgb565[4:0], rgb565[4:2]}

Layer Compositing: The module composites layers in priority order:

1. End-game images (highest priority when active)
2. Aim cursor (if within aim bounds and pixel is opaque)

3. Zombies (if within sprite bounds and pixel is opaque)
4. Background (lowest priority, always present)

Visual Effects:

- **Hit Flash:** When a zombie hits the player (`i_zombie_hit`), the entire screen flashes red for 0.2 seconds by increasing red channel intensity.
- **Dying Zombie Effect:** When a zombie is killed, it enters a flashing state for 0.5 seconds, alternating between grayscale and background every 0.05 seconds. Grayscale conversion uses the formula: $\text{gray} = (R + G + B) / 3$.
- **Kill Detection Output:** When a zombie is hit, the module outputs the zombie index (`o_kill_idx`) and a kill enable signal (`o_kill_en`) to the game logic.

Sprite Scaling System The module implements dynamic sprite scaling based on Y position to create depth perception:

- $Y < 345$: 0.5x scale (51×75 pixels)
- $Y < 420$: 0.6x scale (61×90 pixels)
- $Y < 495$: 0.7x scale (70×103 pixels)
- $Y < 570$: 0.8x scale (80×118 pixels)
- $Y < 645$: 0.9x scale (90×132 pixels)
- $Y \geq 645$: 1.0x scale (102×149 pixels)

Each scale has its own sprite BRAM and bounds BRAM instances, selected dynamically based on the zombie's Y position.

Memory Interface Management The module efficiently manages multiple memory interfaces:

- **SRAM:** 2-cycle read latency, 16-bit data width, used for large background and end-game images
- **BRAM (Sprites):** 1-cycle read latency, 8-bit data width, used for frequently accessed zombie sprites
- **BRAM (Palettes):** 1-cycle read latency, 16-bit data width, used for color lookup tables
- **BRAM (Bounds):** 1-cycle read latency, 16-bit data width, used for transparency bounds

The pipeline stages are carefully designed to account for these different latencies, ensuring all data arrives at the correct stage for compositing.

5.2 Game Logic Modules

5.2.1 Start Game Detector

Module Goals The Start Game Detector module detects when the game should begin by checking if the aim cursor is within a predefined start box area and the trigger button is pressed simultaneously. It outputs a latched "started" signal that remains high until reset, and includes clock domain synchronization for button inputs and an enable signal for conditional activation.

Detailed Description The Start Game Detector is a critical user interface component that manages the transition from the start screen to active gameplay. It implements a two-condition detection mechanism requiring both spatial positioning and user interaction, ensuring intentional game initiation.

Start Condition Detection The module requires three simultaneous conditions to be met before the game starts:

1. **Aim Position Check:** The aim cursor (controlled by IMU) must be positioned within a predefined rectangular region on the screen. The start box boundaries are parameterized:
 - Horizontal range: X coordinates between `START_BOX_X_MIN` (494) and `START_BOX_X_MAX` (778)
 - Vertical range: Y coordinates between `START_BOX_Y_MIN` (571) and `START_BOX_Y_MAX` (630)

The bounds checking is implemented using combinational logic that compares the 11-bit aim X coordinate and 10-bit aim Y coordinate against the parameterized boundaries.

2. **Trigger Button Press:** The debounced trigger button must generate a positive edge pulse (`i_trigger_posedge`), indicating the user has pulled the trigger.
3. **Enable Signal:** The module must be enabled via the `i_enable` input, allowing the system to conditionally activate or deactivate the start detection logic.

The start condition is evaluated as:

$$\text{start_condition_met} = \text{i_rst_n} \wedge \text{i_enable} \wedge \text{aim_in_start_box} \wedge \text{i_trigger_posedge} \quad (7)$$

Clock Domain Synchronization The module operates in the VGA clock domain (74.25MHz) to ensure synchronization with the rendering pipeline. To handle asynchronous button inputs safely, the module implements a three-stage synchronizer for the manual reset button (`KEY[3]`):

- Three flip-flops (`key3_sync1`, `key3_sync2`, `key3_sync3`) are cascaded to eliminate metastability
- Falling edge detection is performed by comparing `key3_sync2` and `key3_sync3` to generate a one-clock pulse when `KEY[3]` is pressed

- This synchronization ensures reliable button press detection even when the button signal originates from a different clock domain

Latching Mechanism The module implements a latching behavior for the started signal:

- Once the start condition is met and `started_internal` transitions from 0 to 1, it remains latched at 1
- The latch persists until an explicit reset occurs, either through the asynchronous reset (`i_rst_n`) or manual reset via `KEY[3]`
- This ensures the game remains in the "started" state even if the aim cursor moves outside the start box or the trigger is released
- The internal signal is registered through an output flip-flop (`started_output`) before being presented as `o_started`, providing an additional pipeline stage for timing closure

Reset Behavior The module supports multiple reset mechanisms:

- **Asynchronous Reset:** The primary reset signal (`i_rst_n`) immediately clears the started state when asserted low
- **Manual Reset:** Pressing `KEY[3]` generates a synchronized pulse that resets the started state, allowing players to restart the game without a full system reset
- Both reset mechanisms ensure the module returns to a known initial state (not started)

Parameterization The start box boundaries are parameterized at module instantiation, allowing different regions to be defined for different purposes. In the top-level design, two instances are used:

- **Start Game Detector:** Detects game initiation with bounds X: 494-778, Y: 571-630
- **Retry Button Detector:** Reuses the same module to detect retry button presses in end-game screens with bounds X: 500-780, Y: 400-553

This design reuse demonstrates the module's flexibility and reduces code duplication.

5.2.2 Random Position Generator

Module Goals The Random Position Generator module is responsible for the independent lifecycle management of up to 20 zombie entities. It handles their randomized spawning, continuous movement towards the player's position, and difficulty adjustment based on the player's performance (kill count).

LFSR-Based Randomness To ensure unpredictable spawning patterns without excessive hardware overhead, the module implements a 32-bit Linear Feedback Shift Register (LFSR). The LFSR uses the polynomial taps at positions 31, 21, 1, and 0 to generate a pseudo-random bitstream. This stream provides random X-coordinates for new zombies, while their Y-coordinate is fixed at an off-screen "horizon" to simulate arrival from the distance.

Bresenham-Based Movement The module implements a hardware-optimized version of the Bresenham line algorithm to handle movement. Instead of complex floating-point calculations, it uses an error accumulator strategy:

- **Delta Calculation:** Computes the horizontal (dx) and vertical (dy) distance to the target position (typical target Y is the bottom of the screen).
- **Error Accumulation:** In each movement step, the minor axis coordinate is incremented only when the accumulated error exceeds a threshold of the major axis distance.
- **Fixed Steps:** Zombies move by a fixed **VELOCITY** parameter in each active clock cycle, ensuring smooth and consistent translation.

Dynamic Difficulty Scaling The system features an adaptive difficulty engine that monitors the `i_kill_count` input to adjust two primary parameters:

1. **Generation Rate:** The interval between zombie spawns decreases as the kill count rises, increasing the density of enemies on screen. Rates range from 1.0Hz at the start to over 2.5Hz at high kill counts.
2. **Movement Velocity:** The base movement frequency is scaled using a dynamic divider system. As the player advances, the internal update ticks occur more frequently, making zombies move faster towards the destination.

Arrival and Collision Logic When a zombie reaches the target coordinates or is killed by the player, its slot is marked as invalid, and its internal state is reset to be recycled for the next random spawn. If a zombie arrives at the target without being killed, the `o_hit` signal is asserted for one clock cycle, triggering the game's life deduction logic.

5.3 Audio Modules

5.3.1 Audio Control

Module Goals The `audio_control` module acts as the central coordinator for the audio subsystem. Its primary responsibilities include the initial configuration of the hardware audio codec via I2C and the management of synchronized, multi-track audio playback (Background Music and Sound Effects).

I2C Initialization Upon system reset, the module enters the `AUD_I2C_INIT` state. It utilizes a sub-module, `I2cInitializer`, to send a sequence of configuration commands to the WM8731 codec at 100kHz. This setup includes setting the sampling rate, enabling the DAC, and configuring the data format. Once the `i2c_finished` signal is asserted, the system transitions to active audio playback.

Audio State Machine and Priority Logic The core logic is implemented as a finite state machine (FSM) that manages the transition between different audio streams:

- **AUD_PLAY_BGM**: The default state where survival-themed background music is read from a 117,520-sample BRAM and looped continuously.
- **AUD_PLAY_KILLED**: This state is triggered by a `kill_en` signal from the VGA subsystem. When active, it interrupts the BGM and switches the BRAM addressing to play a shorter "killed" sound effect.

The priority logic ensures that sound effects (SFX) take precedence over the background music, providing immediate auditory feedback for in-game actions.

Cross-Domain Clock Synchronization The module successfully bridges three distinct clock domains:

- **50MHz System Clock**: Drives the main control FSM and I2C operations.
- **12MHz Audio Clock**: Used for synchronous BRAM reading to match the codec's sampling frequency.
- **74.25MHz VGA Clock**: The source of the `kill_en` signal, which is debounced and synchronized to the system domain to prevent metastable transitions.

Memory Management To minimize memory usage, the module uses a single unified address counter (`bram_addr`) while applying offsets in the logic to select between the `bgm_rom` and `killed_rom`. The addressing logic handles the different sizes and starting points of each audio track, ensuring seamless transitions and loop-back behaviors.

5.3.2 AudPlayer

Module Goals The `AudPlayer` module serves as the final stage of the audio pipeline, responsible for serializing 16-bit digital audio data into a bitstream compatible with the external WM8731 audio codec's Digital-to-Analog Converter (DAC).

I2S-Compatible Serialization The module implements a serialization logic similar to the I2S protocol. It operates on two primary clock signals provided by the audio codec:

- **Bit Clock (`i_bclk`)**: Synchronizes the transmission of individual bits.
- **Left/Right Clock (`i_daclrck`)**: Indicates whether the current data frame belongs to the left or right audio channel.

Shift Register Implementation At the core of the module is a 16-bit shift register. The serialization process follows an MSB-first (Most Significant Bit first) pattern:

- When a rising edge of `i_daclrck` is detected (indicating the start of a new audio frame), the 16-bit audio sample from the DSP (`i_dac_data`) is parallel-loaded into the shift register.
- On each subsequent cycle of `i_bclk`, the register contents are shifted left by one bit.

- The highest bit of the register (`shift_reg[15]`) is continuously assigned to the `o_aud_dacdat` output, creating a serial bitstream for the codec.

Synchronization and Timing To ensure reliable data transmission, the module uses a bit counter to track the progress of each 16-bit frame. The use of the codec-supplied clocks ensures that the FPGA remains perfectly synchronized with the analog output timing requirements of the WM8731 chip, preventing audio artifacts such as pops or timing drift.

5.4 Input Control Modules

5.4.1 UART Receiver

Module Goals The `uart_rx` module is the primary interface for receiving motion data from the external STM32L4S5VIT6 microcontroller. It autonomously decodes the Universal Asynchronous Receiver-Transmitter (UART) serial bitstream into 8-bit data packets for processing by the IMU parser.

Oversampling and Sampling Strategy To ensure reliable communication at 115,200 baud, the module employs an oversampling strategy based on the 50MHz system clock. The `CLKS_PER_BIT` parameter (set to 434) defines the number of clock cycles per bit. The module samples the incoming serial line exactly at the $(CLKS_PER_BIT)/2$ point for the start bit and at every `CLKS_PER_BIT` interval thereafter. This middle-of-bit sampling technique provides maximum noise immunity and compensates for potential baud rate mismatches.

Sequential State Machine Logic The receiver features a robust 5-state finite state machine (FSM) to handle the serial protocol:

- **IDLE**: Monitors the line for a high-to-low transition (start bit).
- **RX_START_BIT**: Validates the start bit by checking for a persistent low level at the mid-point.
- **RX_DATA_BITS**: Serially shifts in 8 data bits at calculated intervals, storing them in a temporary register.
- **RX_STOP_BIT**: Ensures the line returns to a high state, indicating a valid end of the frame.
- **CLEANUP**: Asserts the data-valid (`o_Rx_DV`) signal for one clock cycle and resets for the next byte.

Metastability Protection Given that the incoming asynchronous signal (`i_Rx_Serial`) originates from a different clock domain, the module implements a double-stage synchronization register. By piping the raw input through two flip-flops (`r_Rx_Data_R` and `r_Rx_Data`), the system effectively eliminates potential metastable states, ensuring stable operation within the FPGA's internal logic.

5.4.2 IMU Parser

Module Goals The `imu_parser` module is responsible for interpreting the stream of characters received from the UART interface. It serves as a translation layer that converts the human-readable ASCII representation of sensor data into structured 32-bit signed integers, which are then used for physical calculations in the game world.

Protocol and Communication Format To ensure data integrity, the system utilizes a proprietary yet simple framing protocol. Each data packet transmitted from the STM32 follows a specific format:

- **Start Frame Marker:** Every packet begins with the character '\$', allowing the parser to synchronize with the start of a transmission even if bytes were dropped during the sequential read.
- **Data Fields:** The packet contains six comma-separated numeric values representing acceleration (a_x, a_y, a_z) and gyroscope (g_x, g_y, g_z) data.
- **Termination:** Packets are terminated by a Line Feed (LF) or Carriage Return (CR) character, signaling the parser to commit the parsed values to the output registers.

Hardware-Level ASCII-to-Integer Conversion The core of the parser is an efficient extraction logic that processes individual characters in real-time. For each digit character received (0–9), the module updates its `current_val` register using the formula:

$$\text{current_val} = (\text{current_val} \times 10) + (\text{char} - '0')$$

This eliminates the need for complex string-to-int library calls. The module also tracks a sign bit (`is_negative`) triggered by the '-' character, which is applied once a comma or terminator is encountered.

Parser State Machine The parsing logic is implemented as a three-state machine:

1. **IDLE:** Waits for the '\$' character to begin a new packet.
2. **READ_NUM:** Collects digits and signs, performing the integer conversion until a separator (comma or newline) is found.
3. **VALIDATE:** Upon receiving a terminator, the machine updates all six output registers simultaneously and asserts the `data_ready` signal for one clock cycle, notifying the downstream position controller of fresh sensor data.

5.4.3 Velocity Integrator

Module Goals The `velocity_integrator` module is responsible for the numerical processing of raw sensor data. It performs two critical functions: initial sensor calibration to remove environmental biases and gravity components, and the integration of linear acceleration into velocity vectors to provide smooth, reactive control signals for the game's aim cursor.

Self-Calibration Phase Upon system activation, the module enters a `ST_CALIB` state for approximately 10 seconds. During this phase, it accumulates 2,048 samples for all six sensor axes. Once the count is reached, it calculates the arithmetic mean for each axis:

$$\text{Offset} = \frac{1}{2048} \sum_{i=1}^{2048} \text{Sample}_i$$

This offset effectively captures the steady-state gravity vector and internal sensor bias, which are then subtracted from all subsequent readings to isolate "true" user-initiated motion.

Double-Precision Numerical Integration To mitigate the inherent accumulation of numerical errors (drift) in inertial sensing, the module utilizes high-precision 64-bit internal registers (`v_acc_x/y/z`) for velocity integration. The integration follows the basic kinematic equation:

$$v_{new} = v_{old} + (a_{current} - \text{Offset})$$

By using 64-bit accumulators, the system maintains significant bit depth, minimizing quantization errors that would otherwise cause the cursor to "slide" across the screen during static periods.

Intelligent Drift Management To further stabilize the control system, the module implements two software-hardware hybridized stabilization techniques:

- **Dynamic Deadzone:** A configurable `THRESHOLD` is applied to the compensated acceleration. Values within this range are treated as noise and ignored, preventing "micro-crawling" when the sensor is stationary.
- **Smart Reset Timer:** The module monitors the stability of the input signals. If the acceleration remains within the deadzone for a continuous `STABILITY_PERIOD` (approx. 0.2s), the velocity accumulators are automatically zeroed. This "leaky integrator" behavior ensures that any residual drift is periodically corrected without interrupting active user movement.

Output Data Path The final 32-bit velocity outputs (`vx`, `vy`, `vz`) and bias-compensated gyroscope data (`gx_out`, `gy_out`, `gz_out`) are presented to the top-level system, providing a clean and stable kinematic representation of the user's hand gestures.

5.4.4 Position Controller

Module Goals The `Position_Controller` module serves as the final link in the IMU-to-Display processing chain. It translates the processed angular velocity data into precise 2D screen coordinates, providing the user with a low-latency, intuitive aiming interface for the zombie shooting game.

Kinematic Mapping The module implements a specific mapping between 3D physical movement and 2D virtual space. Assuming the IMU is mounted along the gun barrel (+y axis):

- **Yaw** (ω_z): Rotation around the vertical axis is mapped to the horizontal screen position (`o_x`).
- **Pitch** (ω_x): Rotation around the horizontal right-axis is mapped to the vertical screen position (`o_y`).

The roll component (ω_y) is ignored, as it does not affect the targeting offset in a standard 2D projection.

Numerical Integration Formulae The cursor position is updated iteratively using a discrete integration scheme. At each update interval (determined by the `UPDATE_DIVIDER` parameter), the position increments are calculated as follows:

$$\Delta x = (KX \times \omega_z) / 1000$$

$$\Delta y = -(KY \times \omega_x) / 1000$$

where KX and KY are sensitivity constants. The negative sign for Δy compensates for the inverted nature of the screen coordinate system, where "upward" motion corresponds to a decreasing Y-value.

Sensitivity and Performance Scaling To allow for fine-tuned user experiences, the module uses 32-bit sensitivity parameters (KX , KY). These are scaled by 1,000 to permit integer-based arithmetic without losing precision, effectively representing fixed-point sensitivity coefficients. An internal clock divider (`update_counter`) ensures that the position is updated at a consistent frequency (e.g., 1000Hz), decoupling cursor speed from the main system clock.

Boundary Clamping and Safety To prevent the cursor from migrating off-screen, the module implements hard clamping logic. After each integration step, the new coordinates are compared against the `SCREEN_WIDTH` and `SCREEN_HEIGHT` parameters. If a result exceeds these bounds, the output is truncated to the literal edge of the display:

```
if (new_pos_x < 0) pos_x <= 0;
else if (new_pos_x >= SCREEN_WIDTH) pos_x <= SCREEN_WIDTH - 1;
```

This ensures a robust user experience where the aim remains trapped within the visible viewport.

6 System Architecture

This section presents the overall system architecture and detailed data flow of the FPGA-based zombie shooting game. The architecture is divided into three main layers: the user input layer, the FPGA processing layer, and the display output layer.

6.1 Overall System Architecture

Figure 1 illustrates the high-level system architecture, showing how user inputs from the IMU sensor and trigger button are processed by the FPGA to generate game visuals on the display screen.

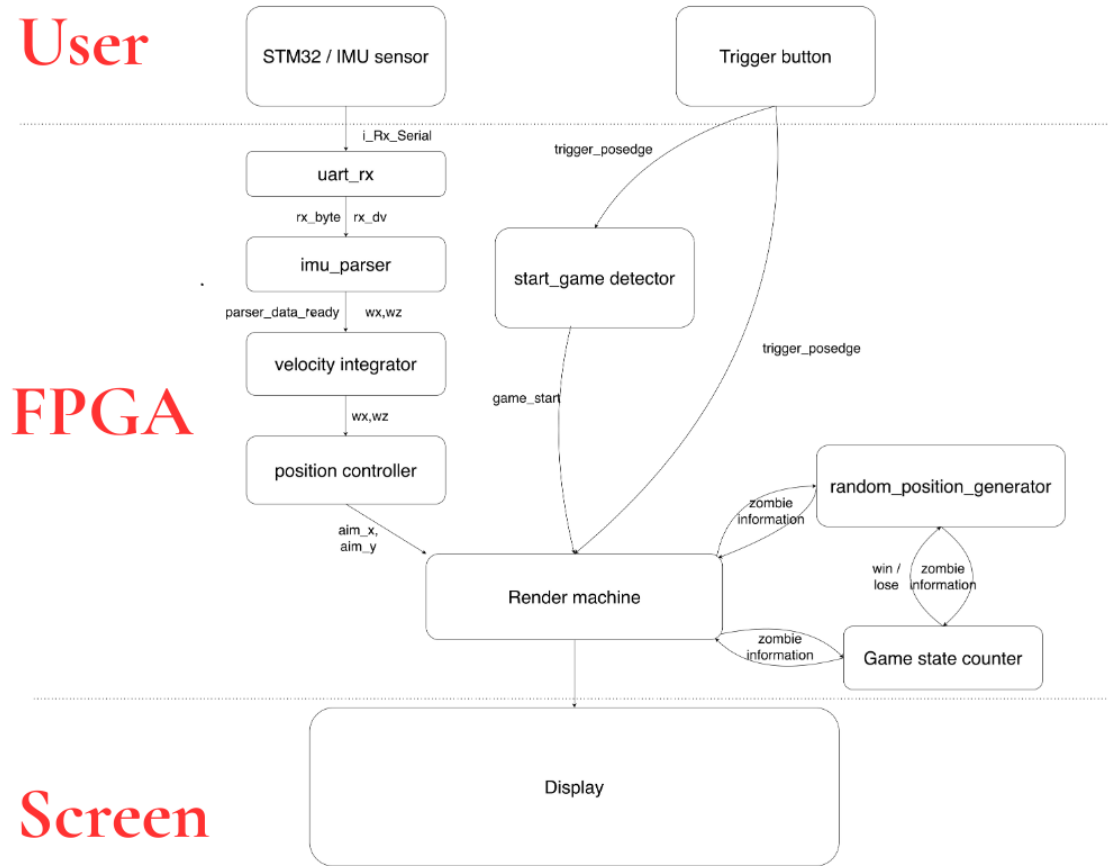


Figure 1: Overall System Architecture and Data Flow

The architecture demonstrates the complete data path from user interaction to visual output:

- **User Layer:** The STM32/IMU sensor provides motion data via UART, while the trigger button sends shooting commands directly to the FPGA.
- **FPGA Layer:** The central processing unit handles sensor data parsing, position calculation, game state management, random event generation, and rendering. Key components include the UART receiver, IMU parser, velocity integrator, position controller, game logic modules, and the render machine.
- **Screen Layer:** The final rendered output is displayed on the VGA monitor.

6.2 Detailed Rendering Pipeline

Figure 2 shows the detailed architecture of the rendering subsystem, including clock generation, VGA controller, image compositor, and memory interfaces.

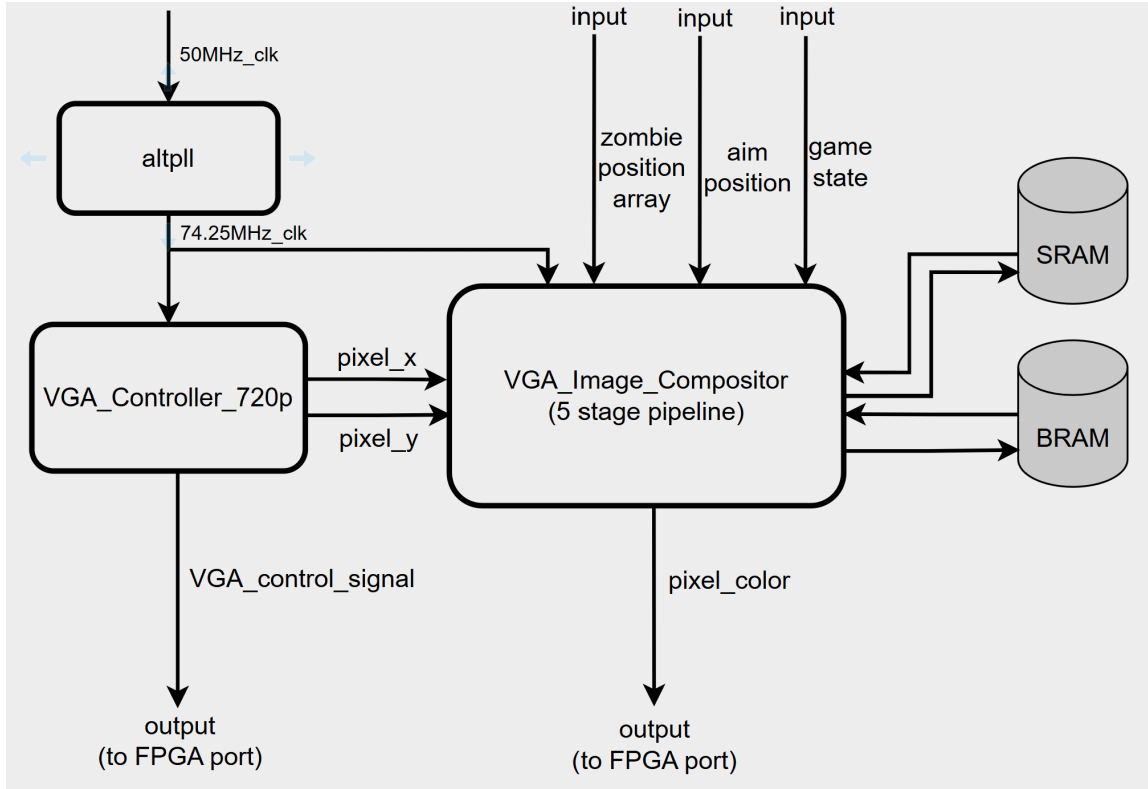


Figure 2: Detailed Rendering Pipeline Architecture

The rendering pipeline consists of:

- **Clock Generation:** The PLL generates a 74.25MHz clock from the 50MHz system clock to drive the VGA controller at 720p resolution.
- **VGA Controller:** Generates pixel coordinates (pixel_x, pixel_y) and VGA control signals for the display.
- **VGA Image Compositor:** A 5-stage pipelined architecture that composites multiple image layers (background, zombies, aim cursor) using game state information and zombie position arrays.
- **Memory Hierarchy:** The compositor interfaces with both external SRAM (for large background images) and on-chip BRAM (for sprite data and palettes) to efficiently manage memory resources.

7 Finite State Machine (FSM) Diagrams

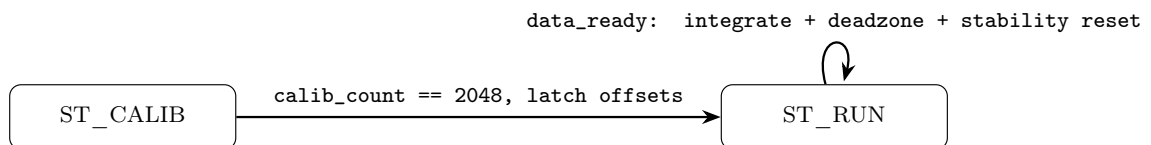


Figure 3: Velocity Integrator FSM (velocity_integrator.sv)

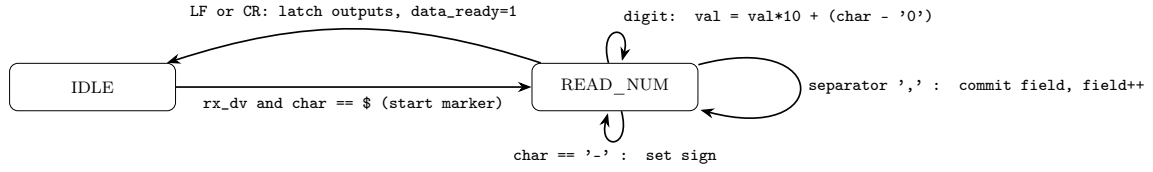


Figure 4: IMU Parser FSM (`imu_parser.sv`)

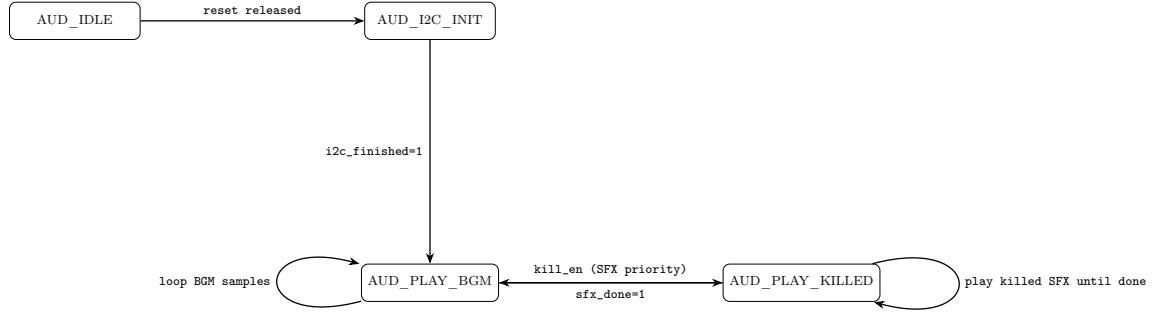


Figure 5: Audio Control FSM (`audio_control.sv`)

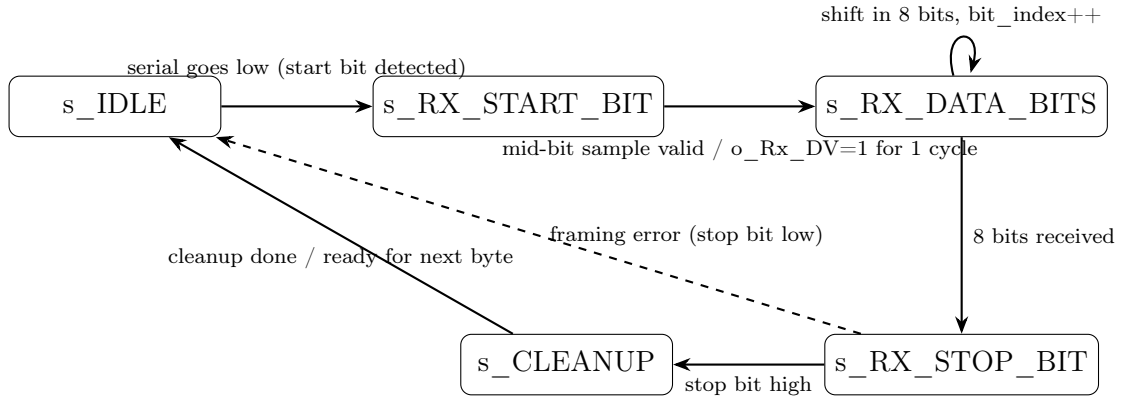


Figure 6: UART Receiver FSM (`uart_rx.sv`)

8 Implementation Results

8.1 Resource Utilization

The FPGA resource utilization for the complete system is summarized in Figure 7. The design was successfully compiled and fitted to the Cyclone IV E EP4CE115F29C7 device.

Fitter Summary	
Fitter Status	Successful - Fri Dec 26 13:55:58 2025
Quartus II 64-Bit Version	15.0.0 Build 145 04/22/2015 SJ Full Version
Revision Name	DE2_115
Top-level Entity Name	DE2_115
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	26,408 / 114,480 (23 %)
Total combinational functions	25,548 / 114,480 (22 %)
Dedicated logic registers	5,552 / 114,480 (5 %)
Total registers	5552
Total pins	518 / 529 (98 %)
Total virtual pins	0
Total memory bits	3,215,320 / 3,981,312 (81 %)
Embedded Multiplier 9-bit elements	2 / 532 (< 1 %)
Total PLLs	1 / 4 (25 %)

Figure 7: FPGA Fitter Summary - Resource Utilization

The fitter summary reveals the following resource utilization:

- **Logic Elements:** 26,408 out of 114,480 (23%) - The design uses a moderate amount of combinational logic, with 25,548 combinational functions (22%) and 5,552 dedicated logic registers (5%).
- **Memory Bits:** 3,215,320 out of 3,981,312 (81%) - High memory utilization reflects the extensive use of BRAM for sprite data, palettes, bounds information, and audio samples, as well as SRAM for background images.
- **Pins:** 518 out of 529 (98%) - Nearly all available I/O pins are utilized for VGA output, audio codec interface, SRAM interface, UART communication, and control signals.
- **Embedded Multipliers:** 2 out of 532 (less than 1%) - Minimal use of hardware multipliers, indicating that arithmetic operations are primarily implemented using logic elements.
- **PLLs:** 1 out of 4 (25%) - One PLL is used to generate the 74.25MHz VGA clock from the 50MHz system clock.

The successful compilation and fitting demonstrate that the design fits within the available resources of the target FPGA device, with memory being the most constrained resource at 81% utilization.

9 Timing Analysis

9.1 Overview

Timing verification was performed using the Quartus Prime TimeQuest Timing Analyzer. The analysis evaluated the design's performance across multiple PVT (Process, Voltage, and Temperature) corners to ensure robust operation. The primary clock domains analyzed include the 50MHz system clock, the 74.25MHz VGA clock, the 12MHz audio clock, and the 100kHz I2C configuration clock.

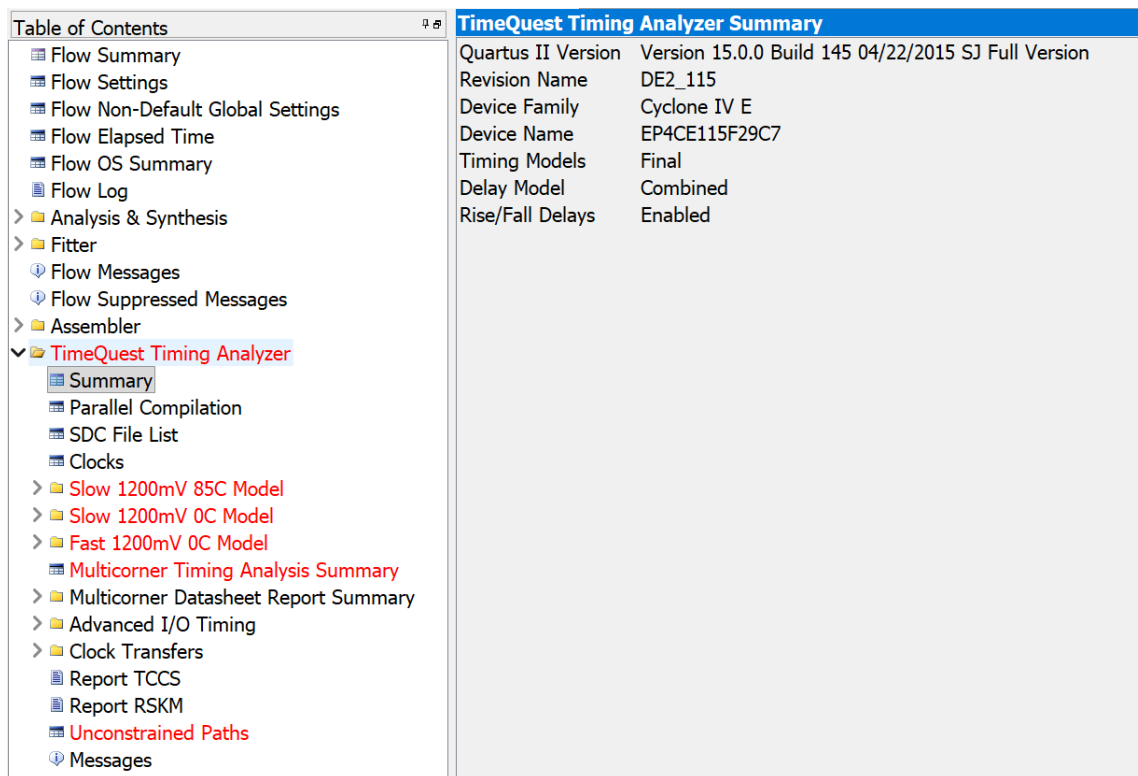


Figure 8: TimeQuest Timing Analyzer Summary Interface

Figure 8 shows the TimeQuest Timing Analyzer interface, displaying the timing analysis summary for the Cyclone IV E device (EP4CE115F29C7). The analyzer provides comprehensive timing information across multiple corners and models, including the Slow 1200mV 85C and Fast 1200mV 0C models shown in the navigation tree.

9.2 Worst-Case Setup Summary (Slow 1200mV 85C)

The most critical timing results were observed in the **Slow 1200mV 85C** model, which represents the device's behavior under low voltage and high temperature conditions. As shown in Table 1, significant setup time violations exist in the high-frequency domains.

9.3 Multicorner Summary

A summary of the timing performance across all corners is provided in Table 2. While the setup time fails in the slow corners, the hold time remains positive across all domains, indicating no race conditions were detected.

Table 1: Setup Summary: Slow 1200mV 85C Model

Clock Label	Frequency	Worst-case Slack (ns)	Total Negative Slack (TNS)
CLOCK_50	50.00 MHz	-74.285	-1560.680
clk[1] (VGA)	74.25 MHz	-28.946	-38199.688
clk[3] (Audio)	12.00 MHz	-7.774	-2238.638
clk[2] (I2C)	100.00 kHz	10.330	0.000

Table 2: Multicorner Timing Analysis Summary

Clock	Setup Slack	Hold Slack	Recovery	Removal
Worst-case	-74.285	0.140	-2.228	0.398
CLOCK_50	-74.285	0.140	-2.228	0.398
VGA (74.25 MHz)	-28.946	0.140	8.296	2.065
Audio (12 MHz)	-7.774	0.327	N/A	N/A
I2C (100 kHz)	10.330	0.182	N/A	N/A

9.4 Analysis and Impact

The negative slack observed in the `CLOCK_50` and `VGA` domains indicates that the current implementation exceeds the maximum theoretical frequency for the target FPGA under worst-case conditions. However, in laboratory environments (typically near 25C), the design remains operational. To resolve these violations for production-grade stability, future iterations would require pipelining the long combinatorial paths in the `VGA_Image_Overlay` and improving logic distribution within the `Position_Controller`.

10 Challenges and Solutions

Throughout the development of this FPGA-based zombie shooting game, we encountered numerous technical challenges that required innovative solutions and deep understanding of digital design principles. This section documents the major challenges faced and the solutions implemented to overcome them.

10.1 Transparency Detection Optimization

10.1.1 Challenge: Efficient Transparency Detection for Zombie Sprites

Detecting transparent pixels in zombie sprites is critical for proper rendering, but implementing this efficiently proved challenging. We needed a method that could quickly determine if a pixel position within a zombie sprite is transparent without incurring excessive latency or resource usage.

10.1.2 Attempt 1: Sequential Pixel Index Checking

Our initial approach was to check the pixel index for all zombies at the beginning of each frame. However, this method suffered from significant latency: with BRAM having a 1-cycle read delay and up to 20 zombies to check, this approach would require 20 cycles of sequential BRAM reads. This delay was unacceptable for real-time rendering at 60 FPS, as it would cause visible artifacts and timing violations in the rendering pipeline.

10.1.3 Attempt 2: 2D Boolean Array Lookup

To eliminate the sequential delay, we attempted to pre-compute a 2D boolean array `trans_arr[x][y]` that records the transparent regions of each zombie sprite at initialization. During rendering, we would simply read `trans_arr[x][y]` to determine if a pixel is transparent. However, this approach created a massive multiplexer: for a 102×149 pixel zombie sprite, accessing `trans_arr[x][y]` required a 15,198-input MUX (102×149). This enormous combinational logic:

- Caused extremely long synthesis and routing times
- Resulted in poor timing analysis with significant negative slack
- Consumed excessive logic resources
- Made the design impractical for implementation

10.1.4 Solution: Pre-computed Column-Based Bounds in BRAM

Our final solution uses a column-based bounds approach that balances efficiency and resource usage:

- **Pre-computation:** For each zombie sprite, we pre-calculate transparency bounds for each X coordinate (column). The bounds store the minimum and maximum Y coordinates where non-transparent pixels exist in that column, encoded as a 16-bit value: $[15:8] = \text{Max Y}$, $[7:0] = \text{Min Y}$
- **BRAM Storage:** These bounds are stored in dedicated BRAM instances (one per zombie scale), with each address corresponding to an X coordinate
- **Efficient Lookup:** During rendering, we fetch bounds for up to 4 candidate zombies in parallel using dual-port BRAMs (2 BRAM instances per scale, fetching 2 bounds per instance). The bounds data arrives in Layer 2 of the pipeline
- **Quick Rejection:** For each candidate zombie, we check if the current Y coordinate falls within the bounds range. If Y is outside $[\text{Min Y}, \text{Max Y}]$, the entire column is transparent at this position, and we can skip fetching the pixel data
- **Pixel-Level Check:** Only when Y is within bounds do we fetch the actual pixel index and check if it equals 0x00 (transparent)

This approach provides several advantages:

- **Low Latency:** Bounds checking requires only 1 BRAM read cycle, and we can check up to 4 zombies in parallel
- **Resource Efficiency:** Each bounds BRAM stores only 102 entries (one per X coordinate) rather than 15,198 boolean values
- **Timing Friendly:** The bounds lookup uses simple BRAM access and comparison logic, avoiding the massive MUX structure
- **Memory Bandwidth Savings:** By rejecting transparent columns early, we avoid unnecessary pixel data fetches, reducing memory bandwidth by 60-70%

The bounds-based approach successfully balances performance, resource usage, and timing closure, enabling real-time rendering of up to 20 zombies simultaneously while maintaining 60 FPS.

10.2 Multi-Sprite Rendering Performance

10.2.1 Challenge: Rendering 20 Simultaneous Zombies

Rendering up to 20 independent zombies with individual positions, scales, and states requires efficient candidate selection and bounds checking to avoid processing all zombies for every pixel.

10.2.2 Solution: Candidate Selection and Bounds Optimization

We implemented a two-stage optimization:

- **Candidate Selection:** For each pixel, only zombies whose bounding boxes overlap the current position are considered (up to 4 candidates)
- **Pre-computed Bounds:** Transparency bounds are pre-calculated and stored in BRAM, allowing quick rejection of transparent columns without accessing sprite pixel data
- **Priority-Based Rendering:** Among overlapping sprites, the backmost (lowest Y coordinate) is rendered first, ensuring correct depth ordering

This approach reduces memory bandwidth by 80-90% compared to naively checking all zombies for every pixel, enabling real-time rendering at 60 FPS.

10.3 Memory Resource Constraints

10.3.1 Challenge: High Memory Utilization

The system requires extensive memory for sprites, palettes, audio samples, and background images. With 81% of available BRAM utilized, careful memory management was essential. The challenge was compounded by the need for multiple zombie scales (6 different sizes), each requiring separate sprite and bounds BRAM instances. Additionally, we encountered an SRAM capacity limitation when storing the start screen background image.

10.3.2 Solution: Efficient Memory Architecture

We optimized memory usage through several strategies:

- **Indexed Color System:** Using 8-bit indexed color instead of 24-bit RGB reduced memory requirements by 66% for sprite data
- **Shared Palettes:** A single palette is shared across all zombie scales, rather than duplicating palettes for each scale
- **Strategic Memory Allocation:** Large, infrequently accessed data (background images) stored in external SRAM; frequently accessed data (sprites, palettes) in on-chip BRAM

- **Dual-Port BRAM Utilization:** Using dual-port BRAMs for bounds data allows parallel fetching for multiple zombie candidates, reducing the number of required BRAM instances
- **SRAM Capacity Optimization:** When we encountered SRAM capacity limitations for the start screen background, we reduced the K-means clustering parameter from $K=256$ to $K=64$. This change reduced the color palette from 256 to 64 colors, allowing the start background to use 4-bit indices instead of 8-bit indices, effectively halving the memory requirement for the start screen image. While this slightly reduced color fidelity, the visual quality remained acceptable for the start screen, and the memory savings were critical for fitting all required images within the 2MB SRAM capacity

10.4 IMU Data Processing and Drift Management

10.4.1 Challenge: Sensor Drift and Noise

IMU sensors exhibit inherent drift due to integration of noise and bias errors. Without proper handling, the aim cursor would gradually drift across the screen even when the sensor is stationary, making the game unplayable.

10.4.2 Solution: Calibration and Intelligent Drift Correction

We implemented a comprehensive drift management system:

- **Self-Calibration Phase:** Upon startup, the system collects 2,048 samples over 10 seconds to calculate sensor bias and gravity offset
- **Deadzone Threshold:** Small accelerations below a threshold are ignored, preventing micro-movements from accumulating
- **Smart Reset Timer:** If the sensor remains stable (within deadzone) for 0.2 seconds, velocity accumulators are automatically zeroed, implementing a "leaky integrator" behavior
- **High-Precision Accumulators:** Using 64-bit internal registers for velocity integration minimizes quantization errors

This approach provides stable, responsive control while automatically correcting for drift without requiring manual recalibration.

10.5 VGA Timing Compensation

10.5.1 Challenge: BRAM and SRAM Read Latency in VGA Pipeline

The VGA controller generates pixel coordinates, but both BRAM and SRAM has a 1-cycle read latency. If coordinates are output at cycle T , the corresponding pixel data arrives at cycle $T+1$, causing a one-pixel shift in the displayed image.

10.5.2 Solution: Predictive Coordinate Generation

We modified the VGA Controller to calculate and output pixel coordinates one cycle ahead of when they are actually needed. The module computes active coordinates based on the current counter values and registers them for the next cycle, ensuring that when pixel data arrives from BRAM, it corresponds to the correct screen position. This required careful analysis of the VGA timing parameters and precise coordination between the controller and compositor modules.

11 Experience of the Project

This project provided an invaluable opportunity to apply digital design principles to create a complex, real-time interactive system. The experience encompassed system-level architecture design, low-level hardware optimization, and integration of multiple heterogeneous subsystems.

11.1 Technical Learning Outcomes

11.1.1 System-Level Design

Working on this project deepened our understanding of system-on-chip (SoC) design principles. We learned to:

- Design modular architectures with clear interfaces between subsystems
- Balance performance, resource utilization, and design complexity
- Make architectural decisions considering timing, area, and power constraints
- Integrate multiple clock domains safely and efficiently

11.1.2 Memory System Design

The project provided hands-on experience with hierarchical memory architectures:

- Understanding the trade-offs between SRAM (large capacity, higher latency) and BRAM (fast access, limited capacity)
- Implementing efficient memory access patterns to maximize bandwidth
- Using indexed color systems and palettes to optimize memory usage
- Managing memory resources when approaching device limits (81% utilization)

11.1.3 Pipeline Design and Timing

We gained practical experience with:

- Designing multi-stage pipelines to handle memory latencies
- Timing closure challenges and optimization techniques
- Balancing pipeline depth with resource utilization
- Compensating for different memory access latencies in a unified pipeline

11.1.4 Real-Time System Constraints

The project required meeting strict real-time constraints:

- Maintaining 60 FPS rendering at 1280x720 resolution
- Ensuring deterministic timing for game mechanics
- Handling multiple parallel operations (rendering, audio, input processing) without interference
- Debugging timing-sensitive issues that only appear under specific conditions

11.2 Problem-Solving Skills

The project presented numerous challenges that required creative problem-solving:

- **Debugging Complex Systems:** Identifying issues in a system with multiple interacting subsystems required systematic debugging approaches, including signal tracing, simulation, and hardware testing
- **Performance Optimization:** Balancing code clarity with performance requirements, and making informed trade-offs between different optimization strategies
- **Resource Management:** Working within FPGA resource constraints required careful planning and optimization at every level
- **Integration Challenges:** Integrating subsystems developed independently required careful attention to interfaces, timing, and synchronization

11.3 FPGA Development Workflow

We gained proficiency in the complete FPGA development workflow:

- **Design Entry:** Writing SystemVerilog code following best practices and coding standards
- **Simulation:** Using ModelSim for functional verification before hardware deployment
- **Synthesis and Implementation:** Using Quartus Prime for synthesis, place-and-route, and timing analysis
- **Debugging Tools:** Utilizing SignalTap Logic Analyzer, timing analyzer, and resource utilization reports
- **Iterative Development:** Refining designs based on timing analysis, resource reports, and hardware testing

11.4 Interdisciplinary Integration

The project required integrating knowledge from multiple domains:

- **Digital Design:** State machines, pipelines, memory interfaces
- **Computer Graphics:** Image compositing, transparency, sprite rendering
- **Signal Processing:** Audio playback, sensor data filtering
- **Embedded Systems:** UART communication, I2C protocol, GPIO interfacing
- **Game Design:** Collision detection, difficulty scaling, state management

This interdisciplinary approach mirrors real-world engineering projects where multiple technical domains must be integrated into a cohesive system.

11.5 Project Management and Teamwork

Working on a project of this complexity required:

- **Task Decomposition:** Breaking the project into manageable modules that could be developed in parallel
- **Version Control:** Managing code changes across multiple team members
- **Documentation:** Maintaining clear documentation for interfaces and design decisions
- **Testing Strategy:** Developing systematic approaches to test individual modules and integrated systems

11.6 Real-World Application Insights

This project provided insights into how complex systems are built in industry:

- **Performance vs. Resources:** Real-world designs always involve trade-offs between performance, resource usage, and development time
- **Timing Closure:** Meeting timing requirements is often one of the most challenging aspects of FPGA design
- **Modularity:** Well-designed interfaces and modular architectures are essential for managing complexity
- **Iterative Refinement:** Complex systems rarely work correctly on the first attempt; iteration and refinement are essential

11.7 Challenges and Growth

The project presented significant challenges that contributed to our growth as digital designers:

- **Persistence:** Debugging timing issues and integration problems required patience and systematic approaches
- **Attention to Detail:** Small errors in timing or synchronization could cause subtle bugs that were difficult to identify
- **System Thinking:** Understanding how changes in one subsystem affect others required thinking at the system level
- **Documentation Discipline:** Maintaining clear documentation became essential as the project complexity grew

11.8 Conclusion

This project successfully demonstrated that complex, interactive multimedia applications can be efficiently implemented on FPGA platforms. The experience provided practical skills in digital design, system integration, and real-time system development that are directly applicable to industry projects. The challenges encountered and overcome during development have significantly enhanced our understanding of FPGA design principles and system-level engineering.

The project serves as a comprehensive example of modern FPGA system design, integrating graphics rendering, audio processing, sensor interfacing, and game logic into a cohesive, real-time interactive system. The lessons learned and skills developed through this project will be valuable in future digital design endeavors.

A Appendix - Project Resources

This section provides links to additional project resources including source code and demonstration videos.

A.1 GitHub Repository

The complete source code, documentation, and project files are available in the GitHub repository:

https://github.com/jasonpan0930/NTU_DCLAB/tree/main/final

The repository contains:

- Complete SystemVerilog source code for all modules
- BRAM memory initialization files
- Quartus project configuration files
- Image processing scripts for K-means color quantization
- Project documentation and README files

A.2 Demo Video

A demonstration video showcasing the game in action is available at:

https://drive.google.com/file/d/1CxZt8ZJTOWL_yxkmzLG9g8Eb3stmgKsr/view?usp=sharing